https://algs4.cs.princeton.edu

Dynamic Programming

- *introduction*
- *Fibonacci numbers*
- *interview problems*
- *shortest paths in DAGs*

https://algs4.cs.princeton.edu

Dynamic Programming

- *introduction*
- *Fibonacci numbers*
- *interview problems*
- *shortest paths in DAGs*

Dynamic programming overview

Algorithm design paradigm.

- Decompose a complex problem into simpler, overlapping subproblems.
- Build up solutions to progressively larger subproblems.

  (caching results for efficient reuse)

THE THEORY OF DYNAMIC PROGRAMMING

RICHARD BELLMAN

1. **Introduction.** Before turning to a discussion of some representative problems which will permit us to exhibit various mathematical features of the theory, let us present a brief survey of the fundamental concepts, hopes, and aspirations of dynamic programming.

To begin with, the theory was created to treat the mathematical problems arising from the study of various multi-stage decision processes, which may roughly be described in the following way: We have a physical system whose state at any time t is determined by a set of quantities which we call state parameters, or state variables. At certain times, which may be prescribed in advance, or which may be determined by the process itself, we are called upon to make decisions which will affect the state of the system. These decisions are equivalent to transformations of the state variables, the choice of a decision being identical with the choice of a transformation. The outcome of the preceding decisions is to be used to guide the choice of future ones, with the purpose of the whole process that of maximizing some function of the parameters describing the final state.

Examples of processes fitting this loose description are furnished by virtually every phase of modern life, from the planning of industrial production lines to the scheduling of patients at a medical clinic; from the determination of long-term investment programs for universities to the determination of a replacement policy for machinery in factories; from the programming of training policies for skilled and unskilled labor to the choice of optimal purchasing and inventory policies for department stores and military establishments.

Richard Bellman *46

Applications.

- Operations research: multistage decision processes, control theory, optimization, …
- Computer science: AI/ML, compilers, systems, graphics, databases, robotics, theory, …
- Economics.
- Bioinformatics.
- Information theory.
- Tech job interviews.

Bottom line. Powerful and broadly applicable technique.

Classic dynamic programming algorithms

Geometry: De Boor (splines).

Utilities. Unix diff (file comparison).

AI/ML: Viterbi (hidden Markov models).

Computer graphics: Avidan–Shamir (seam carving). ⟵ *see Assignment 6*

Databases: System R algorithm (optimal join order).

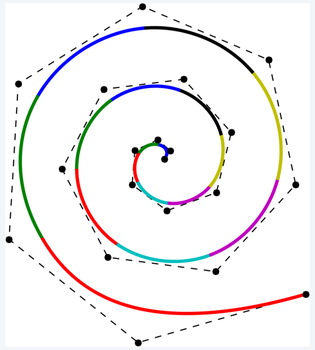
Graph processing: Bellman–Ford–Moore (shortest paths). ⟵ *shortest paths lecture*

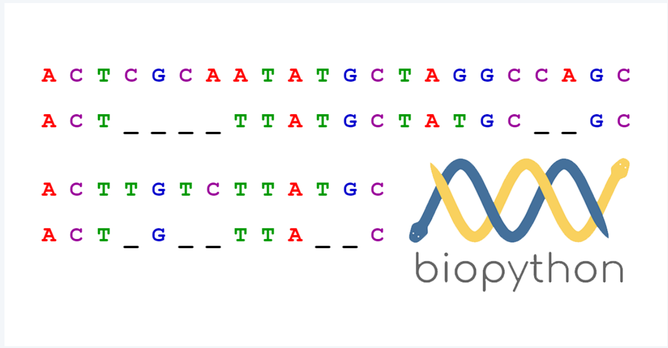
Computational biology: Nedleman–Wunsch, Smith–Waterman (sequence alignment).

Programming languages: Cocke–Kasami–Younger (parsing context-free grammars).

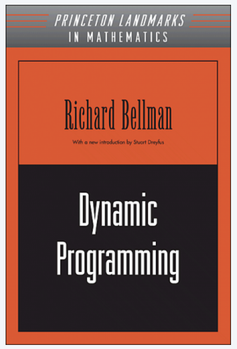
Theory: **NP**-complete graph problems on trees (vertex color, vertex cover, independent set, …).

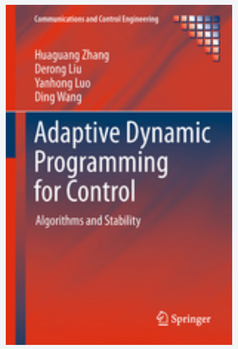
…

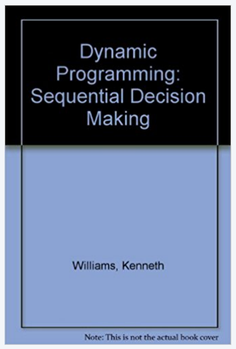


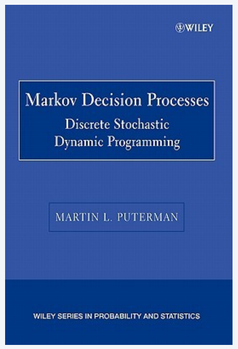


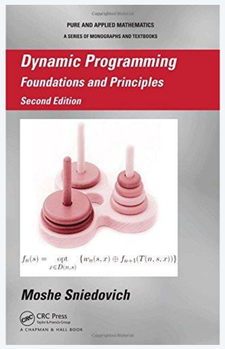
Dynamic programming books

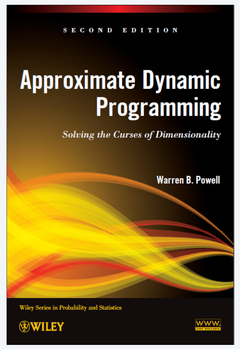


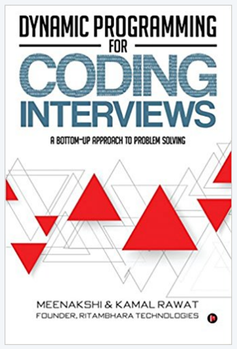


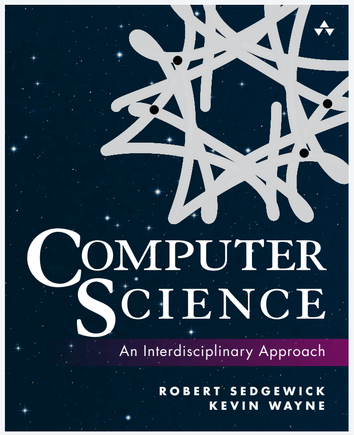




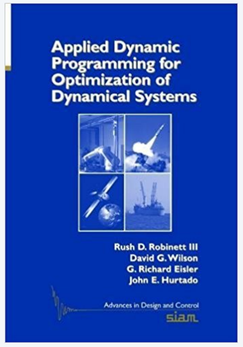


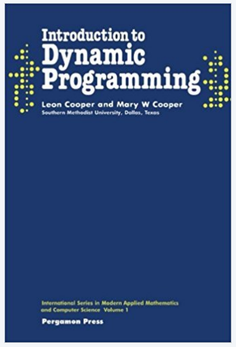


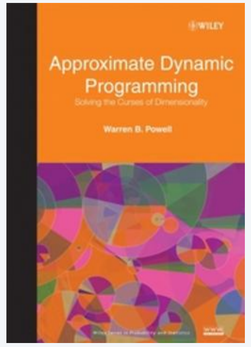


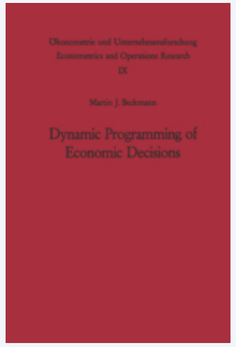


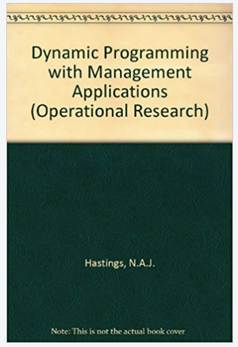
pp. 284–289

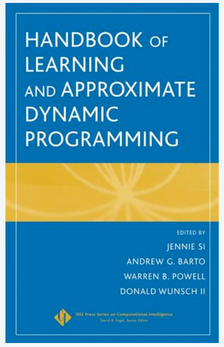


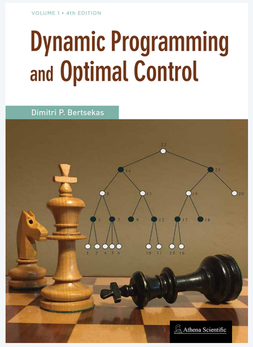


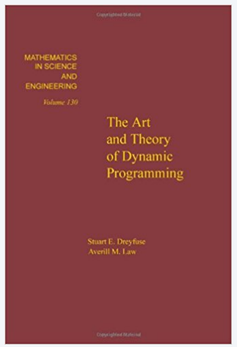


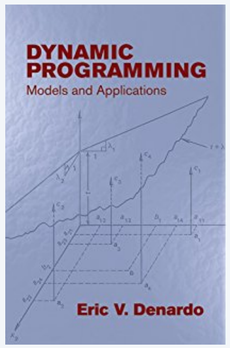


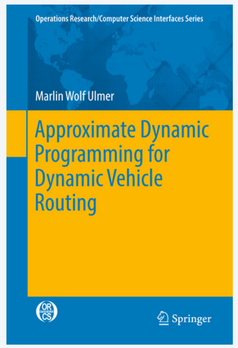


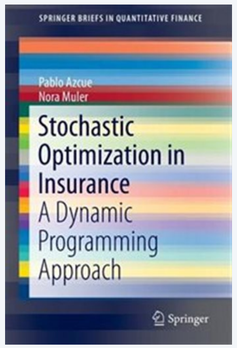


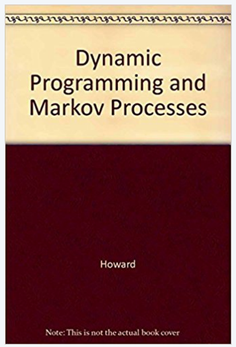


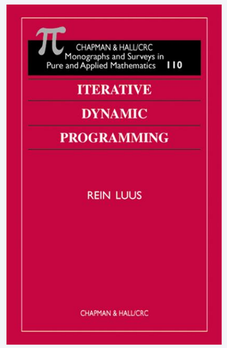


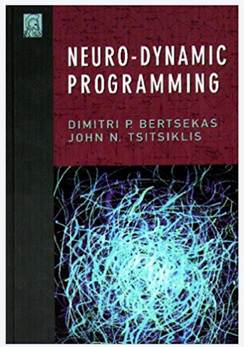












https://algs4.cs.princeton.edu

Dynamic Programming

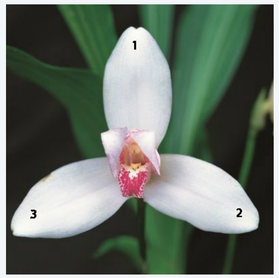
- *introduction*
- *Fibonacci numbers*
- *interview problems*
- *shortest paths in DAGs*

Fibonacci numbers

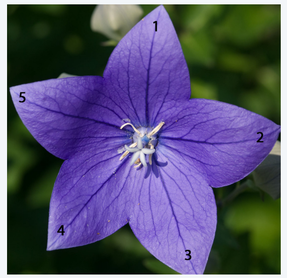
Fibonacci numbers. $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \ldots$

$$F_i = \begin{cases} 0 & \text{if } i = 0 \\ 1 & \text{if } i = 1 \\ F_{i-1} + F_{i-2} & \text{if } i > 1 \end{cases}$$

Leonardo Fibonacci



3



5

8

13

21

34

55

89

Fibonacci numbers: naïve recursive approach

Fibonacci numbers. $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \ldots$

$$F_i = \begin{cases} 0 & \text{if } i = 0 \\ 1 & \text{if } i = 1 \\ F_{i-1} + F_{i-2} & \text{if } i > 1 \end{cases}$$

Goal. Given n, compute F_n .

Direct recursive implementation:

```
public static long fib(int i) {
   if (i == 0) return 0;
   if (i == 1) return 1;
   return fib(i-1) + fib(i-2);
}
```

Dynamic programming: poll 1

How long to compute `fib(80)` using the direct recursive implementation?

直接递归实现

A. Less than 1 second.

B. About 1 minute.

C. More than 1 hour.

D. Overflows a 64-bit `long` integer.

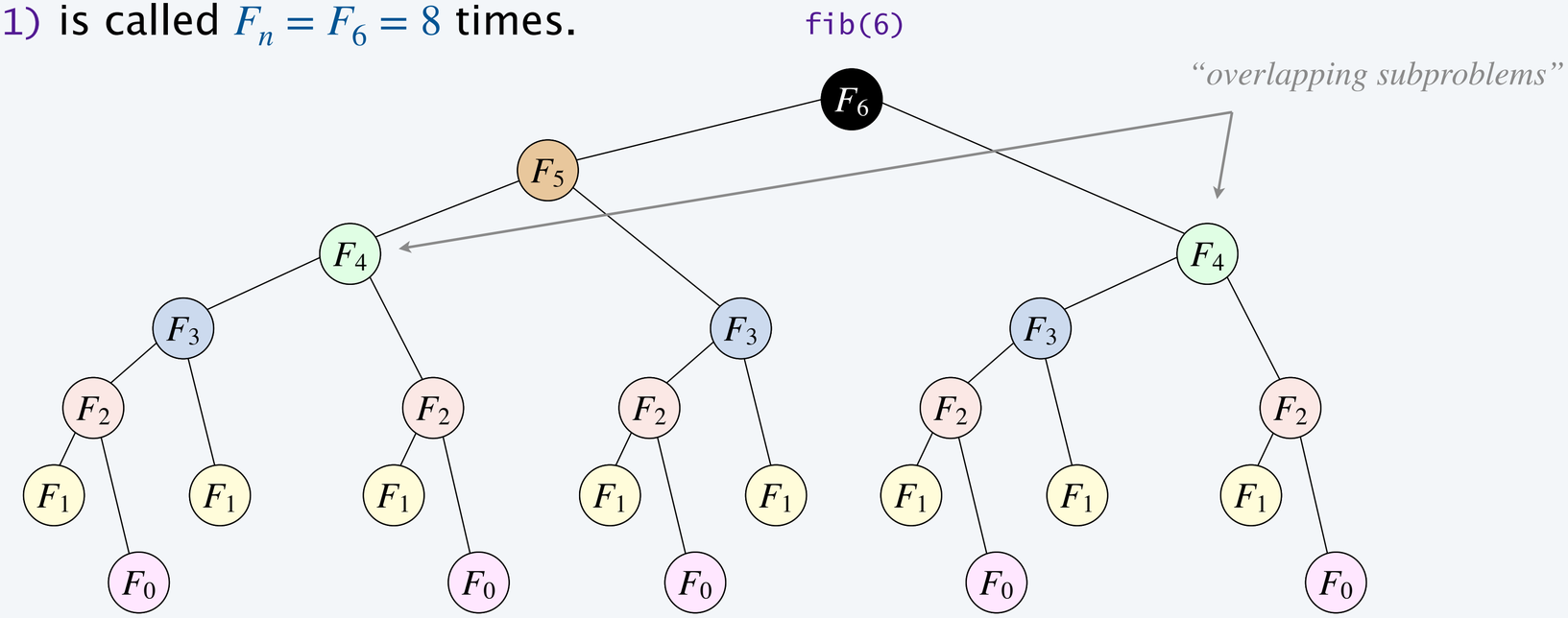
Fibonacci numbers: recursion tree and exponential growth

Exponential waste. Same overlapping subproblems are solved repeatedly.

Ex. When computing `fib(6)`:

- `fib(5)` is called 1 time.
- `fib(4)` is called 2 times.
- `fib(3)` is called 3 times.
- `fib(2)` is called 5 times.
- `fib(1)` is called $F_n = F_6 = 8$ times.

$$F_n \sim \phi^n, \quad \phi = \frac{1+\sqrt{5}}{2} \approx 1.618$$



running time = # subproblems × cost per subproblem

Fibonacci numbers: top-down dynamic programming (memoization)

Memoization.

- Maintain an array (or symbol table) to remember computed values.
- If the value to compute is already known, return it immediately;
  otherwise, compute it; store it; and return it.

```
public static long fib(int i) {
   if (i == 0) return 0;
   if (i == 1) return 1;
   if (f[i] == 0) f[i] = fib(i-1) + fib(i-2);
   return f[i];
}
```

assume global `long` *array* `f[]`,
initialized to 0 (*unknown*)

记忆化：空间换时间

每个子问题计算一次，并且用单位空间存了起来

n个子问题就计算n次，所以时间和空间的开销都是n

Impact. Solves each subproblem F_i only once.

Performance. Computes F_n in $\Theta(n)$ time; uses $\Theta(n)$ extra space.

Fibonacci numbers: bottom-up dynamic programming (tabulation)

自底向上动态规划【迭代】
线性时间，无栈溢出风险【递归有风险】

Tabulation.

- Build computation from the “bottom up.”
- Solve small subproblems first and save their solutions.
- Use those solutions to solve progressively larger subproblems.

```
public static long fib(int n) {
   long[] f = new long[n+1];
   f[0] = 0;
   f[1] = 1;
   for (int i = 2; i <= n; i++)
      f[i] = f[i-1] + f[i-2];
   return f[n];
}
```

smaller subproblems

Impact. Solves each subproblem F_i only once; no recursion.

Performance. Computes F_n in $\Theta(n)$ time; uses $\Theta(n)$ extra space.

Fibonacci numbers: further improvements

Performance improvements.

- Can reduce space by maintaining only two most recent Fibonacci numbers.

```
public static long fib(int n) {
   int f = 0, g = 1;
   for (int i = 0; i < n; i++) {
      g = f + g;
      f = g - f;
   }
   return f;
}
```

f and g are consecutive Fibonacci numbers

因为每次计算新值，只和前面的两个值相关
所以理论上只需要存两个值就可以了

线性时间，常量空间

- Can exploit additional properties of problem:

but, here, our goal is to introduce dynamic programming

$$F_n = \left\lfloor \frac{\phi^n}{\sqrt{5}} \right\rfloor, \quad \phi = \frac{1+\sqrt{5}}{2}$$

$$\begin{pmatrix} F_{i+1} & F_i \\ F_i & F_{i-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^I$$

Dynamic programming recap

Decompose a complex problem into simpler, overlapping subproblems.

[define subproblems: subproblem i = compute Fibonacci number F_i]

Develop a recurrence that expresses larger subproblems in terms of smaller ones.

[easy to solve subproblem i if we know solutions to subproblems $i-1$ and $i-2$]

$$F_i = \begin{cases} 0 & if\ \ i = 0 \\ 1 & if\ \ i = 1 \\ F_{i-1} + F_{i-2} & if\ \ i > 1 \end{cases}$$

Store each subproblem's solution after computing it once.

[store solution to subproblem i in array entry `f[i]`]

Use stored solutions to solve the original problem.

[solution to subproblem n = original problem]

https://algs4.cs.princeton.edu

Dynamic Programming

- *introduction*
- *Fibonacci numbers*
- *interview problems*
- *shortest paths in DAGs*

House painting problem

Goal. Given a row of n black houses, paint some orange so that:

- Maximize total profit, where $profit(i)$ = profit earned for painting house i orange.
- Constraint: no two adjacent houses painted orange.

i	1	2	3	4	5	6
$profit(i)$	10	9	13	20	30	25

profit earned for painting houses 1, 3, and 5 orange
(10+ 13 + 30 = 53)

House painting problem

Goal. Given a row of n black houses, paint some orange so that:

- Maximize total profit, where $profit(i)$ = profit earned for painting house i orange.
- Constraint: no two adjacent houses painted orange.

i	1	2	3	4	5	6
$profit(i)$	10	9	13	20	30	25

profit earned for painting houses 1, 4, and 6 orange
(10+ 20 + 25 = 55)

House painting problem: dynamic programming formulation

Goal. Given a row of n black houses, paint some orange so that:

- Maximize total profit, where $profit(i)$ = profit earned for painting house i orange.
- Constraint: no two adjacent houses painted orange.

Subproblems. $OPT(i)$ = max profit achievable from houses $1, \ldots, i$.

Optimal value. $OPT(n)$.

i	0	1	2	3	4	5	6
$profit(i)$		10	9	13	20	30	25
$OPT(i)$	0	10	10	23	30	53	55

keep house 6 black | *paint house 6 orange*

$$OPT(6) = \max \{ OPT(5), \; profit(6) + OPT(4) \}$$
$$= \max \{ 53, \; 25 + 30 \}$$
$$= 55$$

House painting problem: dynamic programming formulation

Goal. Given a row of n black houses, paint some orange so that:

- Maximize total profit, where $profit(i)$ = profit earned for painting house i orange.
- Constraint: no two adjacent houses painted orange.

Subproblems. $OPT(i)$ = max profit achievable from houses $1, \ldots, i$.

Optimal value. $OPT(n)$.

Binary choice. To compute $OPT(i)$, either:

- Don't paint house i orange: $OPT(i-1)$.
- Paint house i orange: $profit(i) + OPT(i-2)$.

"optimal substructure"
(optimal solution can be constructed from optimal solutions to smaller subproblems)

take best

Dynamic programming recurrence.

$$OPT(i) = \begin{cases} 0 & \text{if } i = 0 \\ profit(1) & \text{if } i = 1 \\ \max\{\, OPT(i-1),\ \ profit(i) + OPT(i-2) \,\} & \text{if } i \geq 2 \end{cases}$$

House painting: naïve recursive approach

Direct recursive implementation:

```
private static int opt(int i, int[] profit) {
   if (i == 0) return 0;
   if (i == 1) return profit[1];
   return Math.max(opt(i-1), profit[i] + opt(i-2));
}
```

Dynamic programming recurrence.

$$OPT(i) = \begin{cases} 0 & \text{if } i = 0 \\ \mathit{profit}(1) & \text{if } i = 1 \\ \max\{\, OPT(i-1),\ \ \mathit{profit}(i) + OPT(i-2) \,\} & \text{if } i \geq 2 \end{cases}$$

Dynamic programming: poll 2

What is running time of the direct recursive implementation as a function of n ?

A. $\Theta(n)$

B. $\Theta(n^2)$

C. $\Theta(c^n)$ for some $c > 1$.

D. $\Theta(n!)$

直接递归需要指数级开销x

```
private static int opt(int i, int[] profit) {
   if (i == 0) return 0;
   if (i == 1) return profit[1];
   return Math.max(opt(i-1), profit[i] + opt(i-2));
}
```

The essence of dynamic programming: remembering the past

“ Those who cannot remember the past are condemned to repeat it. ”

— Dynamic Programming

(*Jorge Agustín Nicolás Ruiz de Santayana y Borrás*)

Housing painting: bottom-up implementation

Bottom-up DP implementation.

```
int[] opt = new int[n+1];
opt[0] = 0;
opt[1] = profit[1];
for (int i = 2; i <= n; i++) {
   opt[i] = Math.max(opt[i-1], profit[i] + opt[i-2]);
}
```

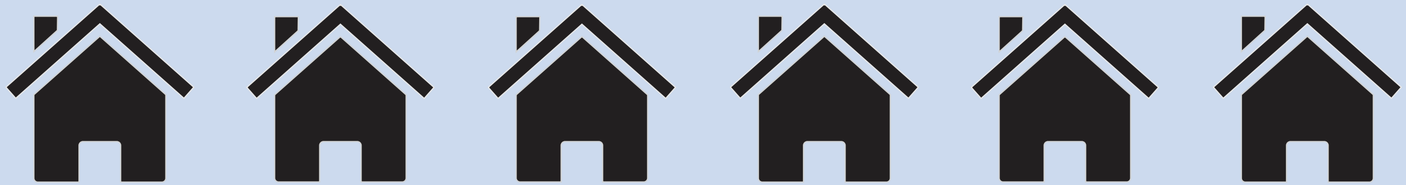
solutions to smaller subproblems already available

$$OPT(i) = \begin{cases} 0 & \text{if } i = 0 \\ profit(1) & \text{if } i = 1 \\ \max\{\, OPT(i-1),\ \ profit(i) + OPT(i-2) \,\} & \text{if } i \geq 2 \end{cases}$$

Proposition. Computing $OPT(n)$ takes $\Theta(n)$ time and uses $\Theta(n)$ extra space.

Housing painting: trace

Bottom-up DP implementation trace.



i	0	1	2	3	4	5	6
$profit(i)$	-	10	9	13	20	30	25
$OPT(i)$	0	10	10	23	30	53	55

OPT(i) = max profit achievable from houses 1, 2, …, i

Housing painting: traceback

Q. We computed the optimal value. How to reconstruct an optimal solution?

A. Retrace optimal choices, starting from optimal value and following choices that led to it.

record these choices while computing the optimal value

i	0	1	2	3	4	5	6
$profit(i)$	–	10	9	13	20	30	25
$OPT(i)$	0	10	10	23	30	53	55
$choice(i)$		*orange*	*black*	*orange*	*orange*	*orange*	*orange*

回溯：不是直接便利choice数组！从最后一个开始往回走
如果此时是orange，就找n-2
反之，找n-1

再开一个choice空间存中间结果便于回溯

choice(i) = color to paint house i that maximizes total profit achievable from houses 1, 2, …, i

Coin changing problem

Problem. Given n coin denominations $\{ d_1, d_2, \ldots, d_n \}$ and a target value V, find the fewest coins needed to make change for V (or report impossible).

Ex. Coin denominations $= \{1, 10, 25, 100\}$, $V = 131$.

Greedy (8 coins). $131¢ = 100 + 25 + 1 + 1 + 1 + 1 + 1 + 1$.

Optimal (5 coins). $131¢ = 100 + 10 + 10 + 10 + 1$.

huí shuō

8 coins
(131¢)

5 coins
(131¢)

vending machine
(out of nickels)

Remark. Greedy algorithm is optimal for U.S. coin denominations $\{1, 5, 10, 25, 100\}$. ← *stay tuned* (*Algorithm Design lecture*)

Dynamic programming: poll 3

Which subproblems for coin changing problem?

A. $OPT(i)$ = fewest coins needed to make change for target value V using only coin denominations $d_1, d_2, \ldots, d_i$.

B. $OPT(v)$ = fewest coins needed to make change for amount v, for $v = 0, 1, \ldots, V$.

C. Either A or B.

D. Neither A nor B.

理解方法：
尝试写出递推式，子问题定义的合理性就很明显了
尤其关注OPT括号内的元素，是不是和定义的目标直接关联
尤其是关注定义的OPT（x）能否有效定义OPT（x-k）的含义
对于B选项，OPT（v） = min（OPT（v-d）+1，...）【基于更少金额的最少钱币数量计算当前v的钱币】
对于A选项，OPT（i-t）没法有效定义，其就相当于是用少于原问题的钱币种类进行定义，已经破坏了问题本身

Coin changing: dynamic programming formulation

Problem. Given n coin denominations $\{ d_1, d_2, \ldots, d_n \}$ and a target value V, find the fewest coins needed to make change for V (or report impossible).

Subproblems. $OPT(v)$ = fewest coins needed to make change for amount v.

Optimal value. $OPT(V)$.

Ex. Coin denominations $\{ 1, 5, 8 \}$ and $V = 10$.

v	0¢	1¢	2¢	3¢	4¢	5¢	6¢	7¢	8¢	9¢	10¢
$OPT(v)$	0	1	2	3	4	1	2	3	1	2	2
$choice(v)$	–	*penny*	*penny*	*penny*	*penny*	*nickel*	*penny*	*penny*	*8-cent*	*penny*	*nickel*



$$\begin{aligned} OPT(10) &= \min \{ 1 + OPT(10-1),\ 1 + OPT(10-5),\ 1 + OPT(10-8) \} \\ &= \min \{ 1 + 2,\ 1 + 1,\ 1 + 2 \} \\ &= 2 \end{aligned}$$

Coin changing: dynamic programming formulation

Problem. Given n coin denominations $\{ d_1, d_2, \ldots, d_n \}$ and a target value V, find the fewest coins needed to make change for V (or report impossible).

Subproblems. $OPT(v)$ = fewest coins needed to make change for amount v.

Optimal value. $OPT(V)$.

Multiway choice. To compute $OPT(v)$,

- Select a coin of denomination $d_i \leq v$ for some i.
- Use fewest coins to make change for $v - d_i$.

take best (among all coin denominations)

optimal substructure

Dynamic programming recurrence.

$$OPT(v) = \begin{cases} 0 & \text{if } v = 0 \\ \min\limits_{i \,:\, d_i \leq v} \{ 1 + OPT(v - d_i) \} & \text{if } v > 0 \end{cases}$$

notation: min *is over all coin denominations of value* $\leq v$ (min *is* ∞ *if no such coin denominations*)

Coin changing: bottom-up implementation

Bottom-up DP implementation.

```
int[] opt = new int[V+1];
opt[0] = 0;

for (int v = 1; v <= V; v++) {
   opt[v] = INFINITY;
   for (int i = 1; i <= n; i++) {
      if (d[i] <= v) {
         opt[v] = Math.min(opt[v], 1 + opt[v - d[i]]);
      }
   }
}
```

$$OPT(v) = \begin{cases} 0 & \text{if } v = 0 \\ \min\limits_{i \,:\, d_i \le v} \{\, 1 + OPT(v - d_i) \,\} & \text{if } v > 0 \end{cases}$$

Proposition. DP algorithm takes $\Theta(n\,V)$ time and uses $\Theta(V)$ extra space.

stay tuned
(Intractability lecture)

Note. Technically, running time not polynomial in input size; underlying problem is **NP**-complete.

n, $\log V$

https://algs4.cs.princeton.edu

Dynamic Programming

- *introduction*
- *Fibonacci numbers*
- *interview problems*
- *shortest paths in DAGs*

Shortest paths in directed acyclic graphs: dynamic programming formulation

Problem. Given a DAG with positive edge weights, find shortest path from s to t.

Subproblems. $distTo(v)$ = length of shortest $s \rightsquigarrow v$ path.

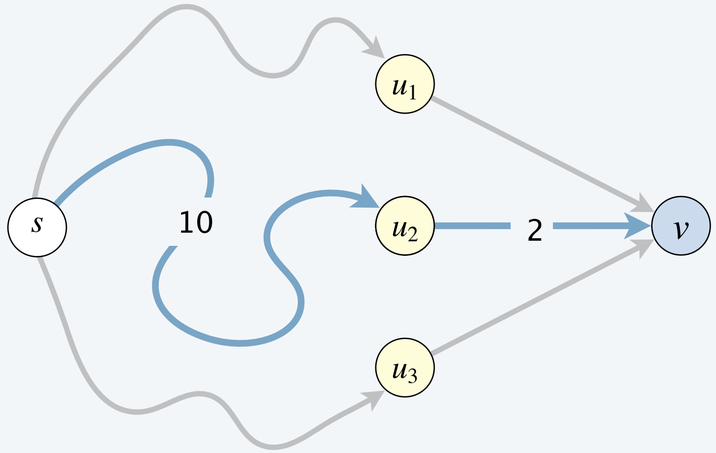
Goal. $distTo(t)$.

Multiway choice. To compute $distTo(v)$:

- Select an edge $e = u \rightarrow v$ entering v.
- Concatenate with shortest $s \rightsquigarrow u$ path.

take best among distTo(u) + weight(e)

optimal substructure



Dynamic programming recurrence.

$$distTo(v) = \begin{cases} 0 & \text{if } v = s \\ \min\limits_{e = u \rightarrow v} \{ distTo(u) + weight(e) \} & \text{if } v \neq s \end{cases}$$

notation: min *is over all edges e that enter v*
(∞ if no such edges)

Dynamic programming: poll 4

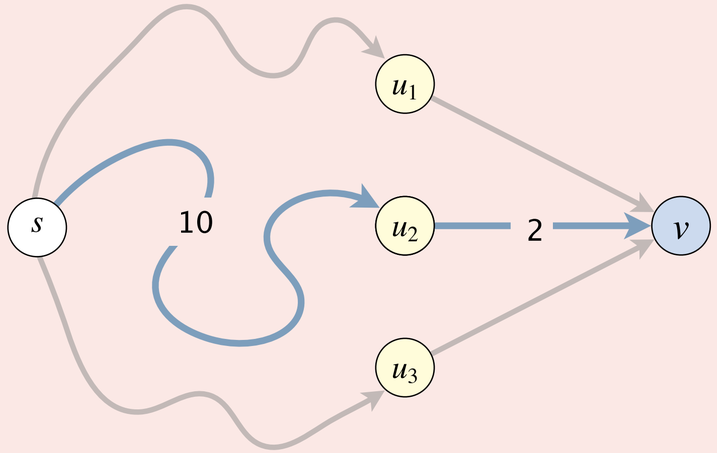
In which vertex order to apply the dynamic programming recurrence?

A. Increasing order of distance from s.

B. Topological order. 拓扑序

C. Reverse topological order.

D. All of the above.



$$distTo(v) = \begin{cases} 0 & \text{if } v = s \\ \min\limits_{e = u \to v} \{ distTo(u) + weight(e) \} & \text{if } v \neq s \end{cases}$$

Shortest paths in directed acyclic graphs: bottom-up solution

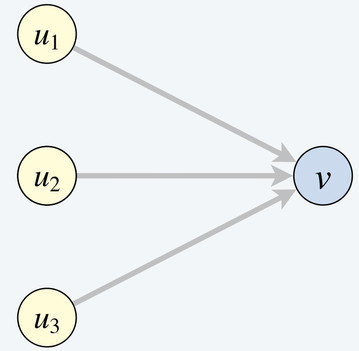
Bottom-up DP implementation. Takes $\Theta(E + V)$ time with two key ideas:

- Solve subproblems in topological order. ← *ensures that $s \rightsquigarrow u_i$ subproblem are solved before $s \rightsquigarrow v$ subproblem*
- Build the reverse digraph G^R. ← *supports efficient access to all of a vertex's incoming edges*

Equivalent (and simpler) computation: Relax vertices in topological order.

- Updates `distTo[]` values incrementally, as in Dijkstra/Bellman-Ford.
- Propagates information along outgoing edges.

```
Topological topological = new Topological(digraph);
for (int v : topological.order())
   for (DirectedEdge e : digraph.adj(v))
      relax(e);
```

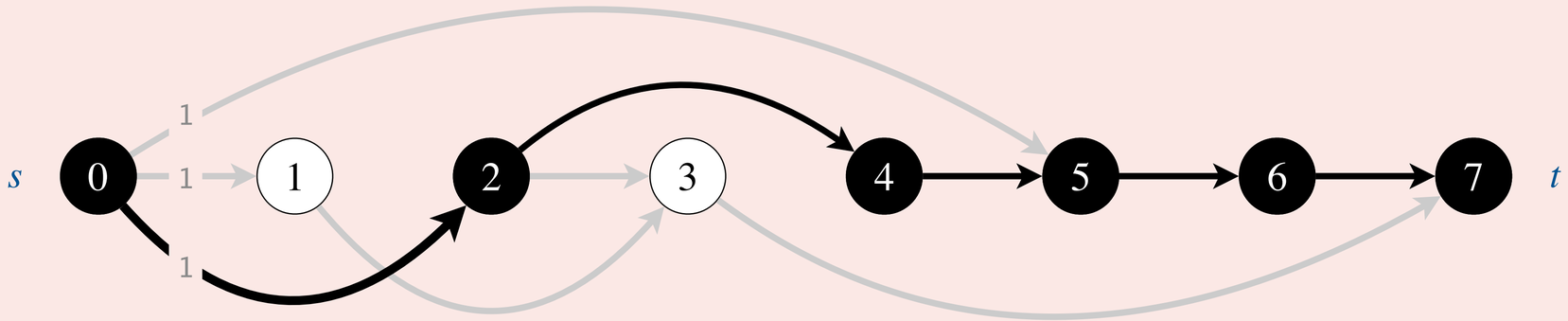


relax vertices u_1, u_2, and u_3 before vertex v

Recovering paths. Maintain `edgeTo[]` array to reconstruct the shortest $s \rightsquigarrow v$ paths.

Dynamic programming: poll 5

Given a DAG, how to find longest path from s to t in $\Theta(E + V)$ time?



longest path from s to t in a DAG (all edge weights = 1)

最长路径算法可轻松转化得到

将边的权值取负

A. Negate edge weights; use DP algorithm to find shortest path.

B. Replace *min* with *max* in DP recurrence.

C. Either A or B.

D. No poly-time algorithm is known (**NP**-complete).

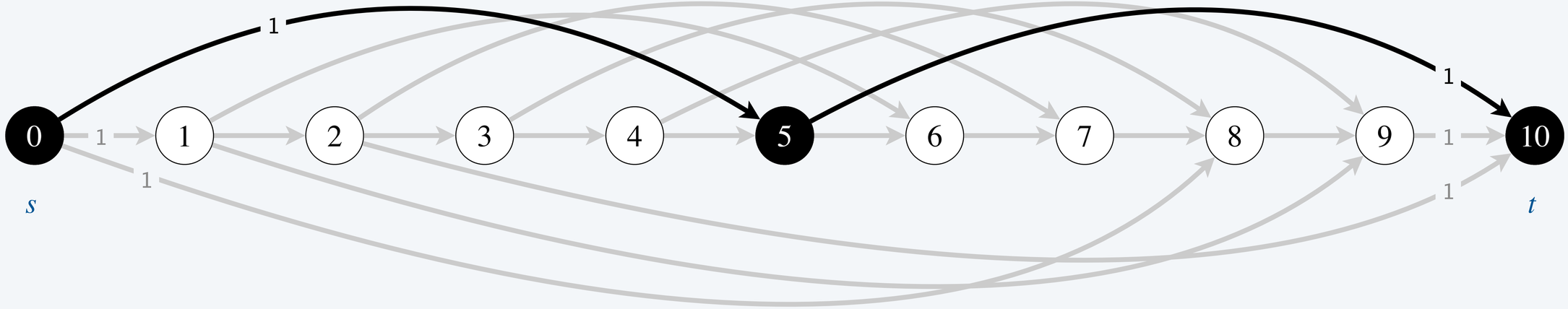
Shortest paths in DAGs and dynamic programming

最短路径问题和钱币兑换问题本质上是同一个问题
算法是众多实际问题的抽象

DP subproblem dependency digraph.

- Vertex v corresponds to subproblem v.
- Edge $v \rightarrow w$ means subproblem v must be solved before subproblem w.
- Digraph must be a DAG. Why?

Ex 1. Modeling the coin changing problem as a shortest path problem in a DAG.



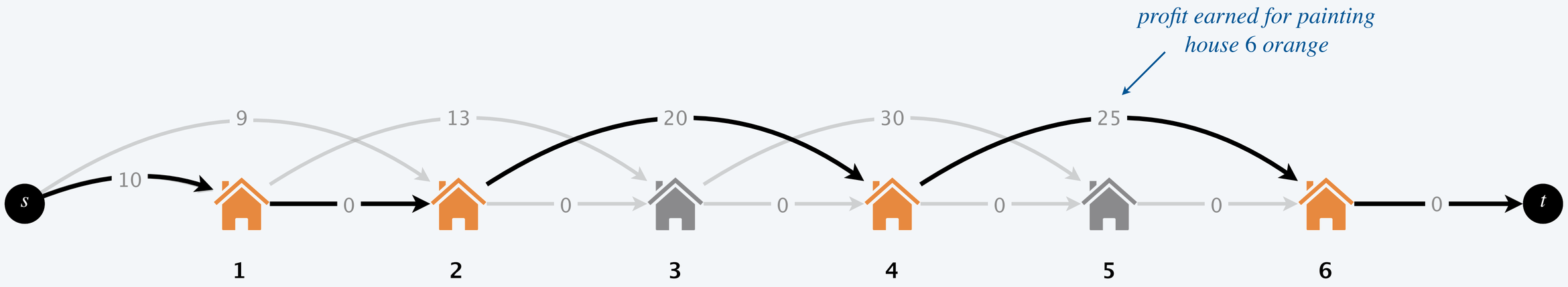
coin denominations = { 1, 5, 8 }, V = 10

Shortest paths in DAGs and dynamic programming

DP subproblem dependency digraph.

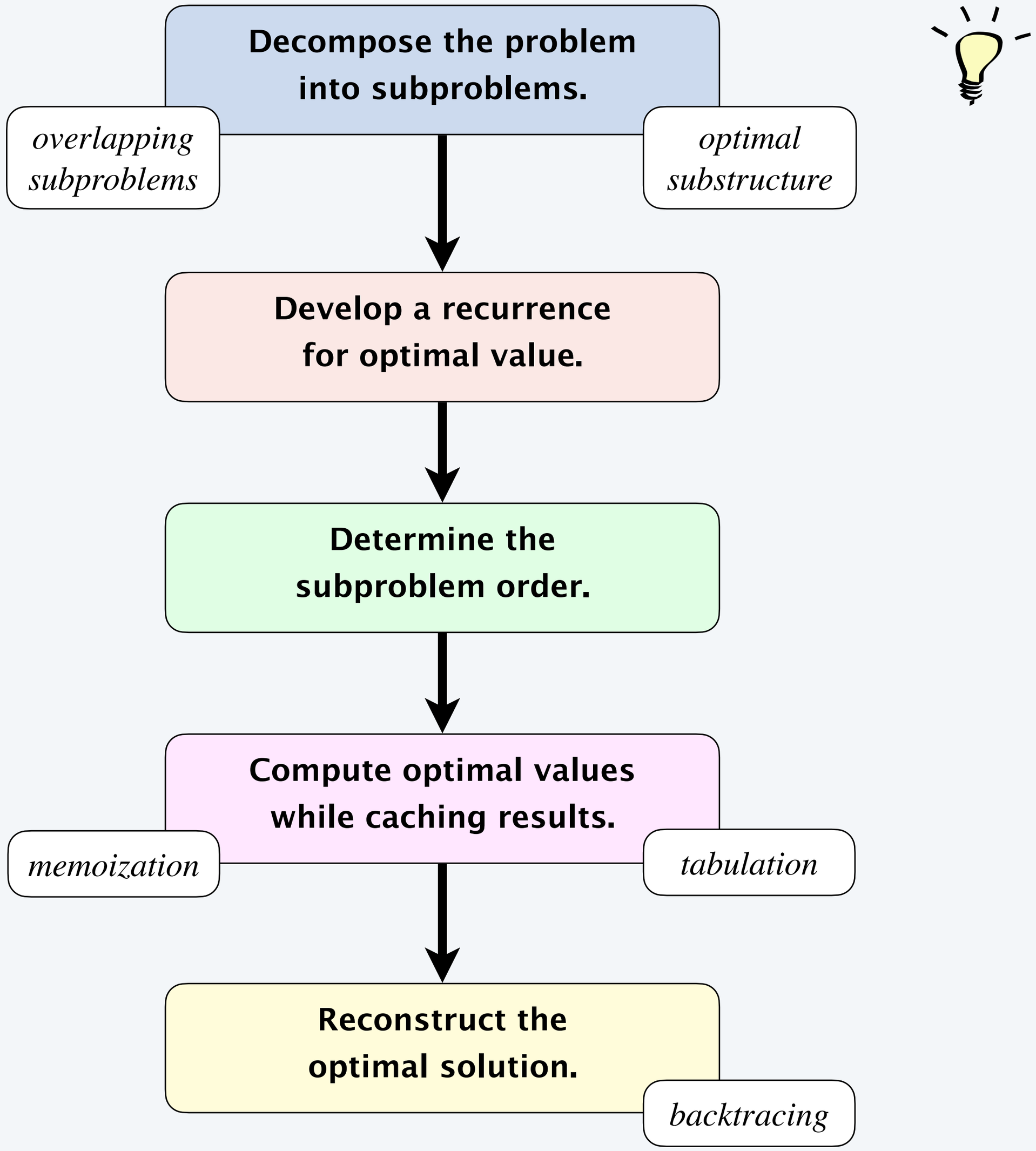
- Vertex v corresponds to subproblem v.
- Edge $v \rightarrow w$ means subproblem v must be solved before subproblem w.
- Digraph must be a DAG. Why?

Ex 2. Modeling the house painting problem as a longest path problem in a DAG.



n = 6; profits = { 10, 9, 13, 20, 30, 25 }

Designing a dynamic programming algorithm



https://algs4.cs.princeton.edu

Dynamic Programming

- *introduction*
- *Fibonacci numbers*
- *interview problems*
- *shortest paths in DAGs*
- *seam carving*

Content-aware resizing

Seam carving. [Avidan–Shamir] Resize an image without distortion for display on cell phones and web browsers.

https://www.youtube.com/watch?v=vIFCV2spKtg

Content-aware resizing

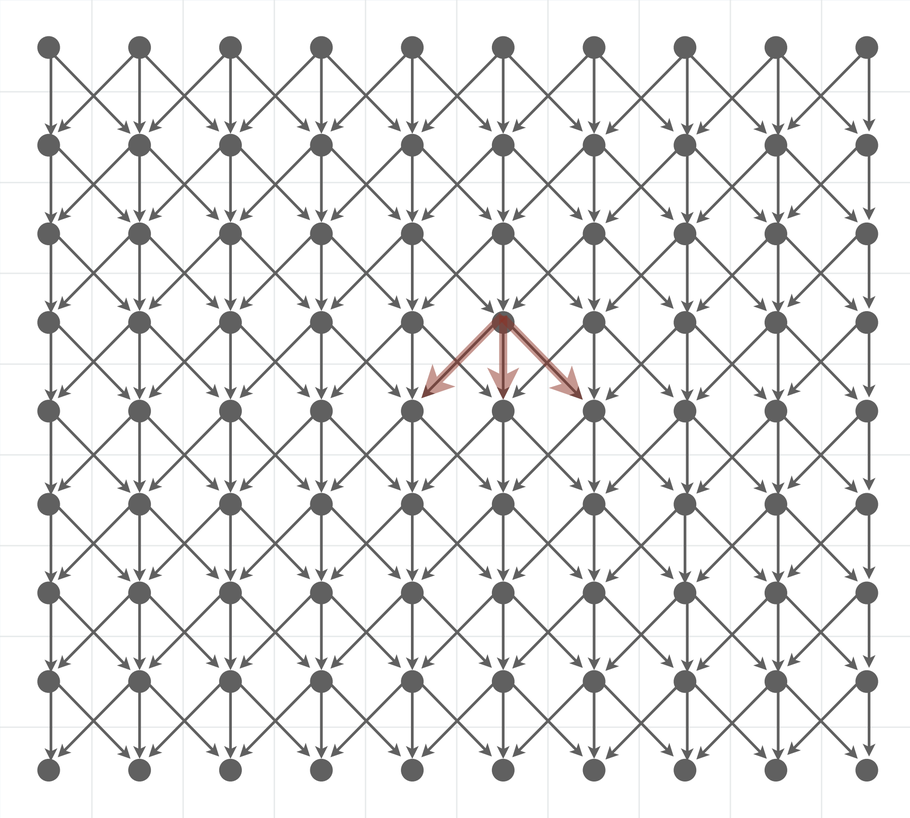
Seam carving. [Avidan–Shamir] Resize an image without distortion for display on cell phones and web browsers.

In the wild. Photoshop, ImageMagick, GIMP, …

Content-aware resizing

To find vertical seam in a picture:

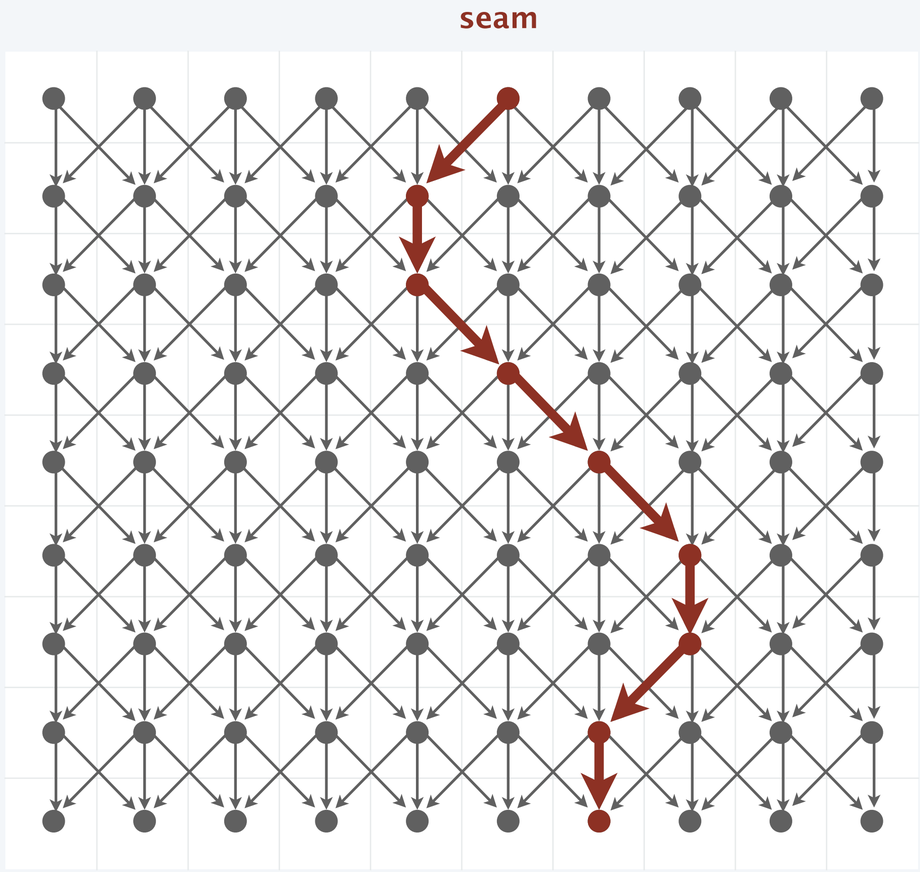
- Grid graph: vertex = pixel; edge = from pixel to 3 downward neighbors (SW, S, SE).
- Weight of pixel = “energy function” of 4 neighboring pixels (N, E, S, W).



Content-aware resizing

To find vertical seam in a picture:

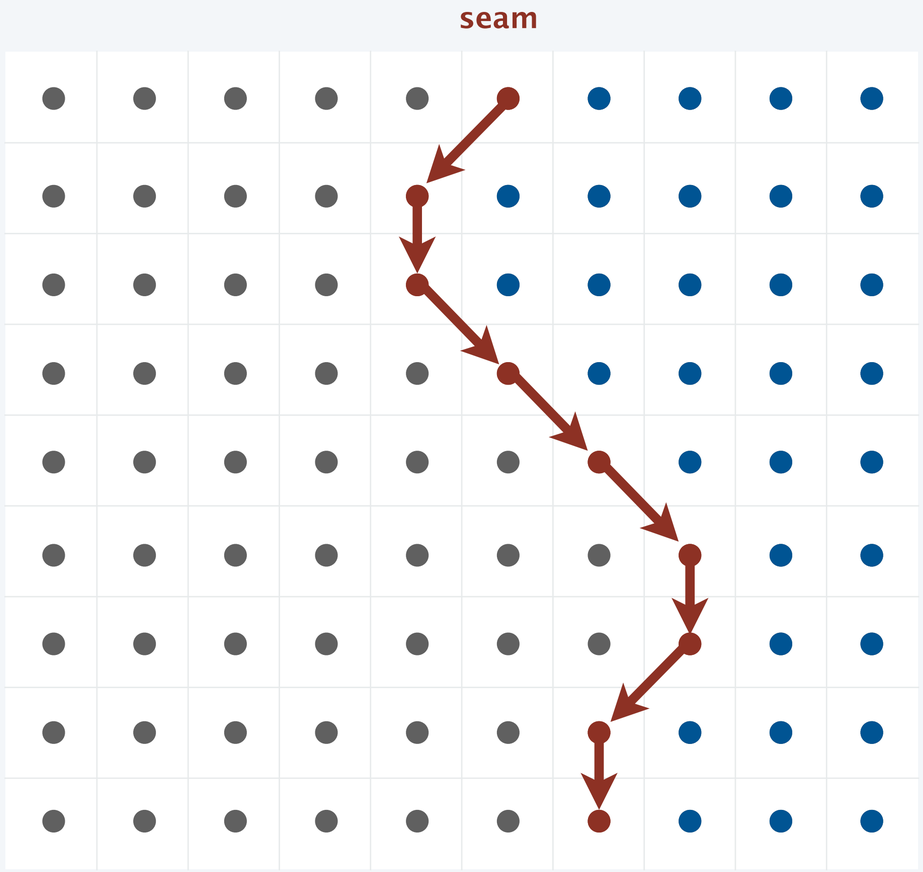
- Grid graph: vertex = pixel; edge = from pixel to 3 downward neighbors (SW, S, SE).
- Weight of pixel = “energy function” of 4 neighboring pixels (N, E, S, W).
- Seam = shortest path (sum of vertex weights) from top to bottom.



Content-aware resizing

To remove vertical seam in a picture:

- Delete pixels on seam (one in each row).



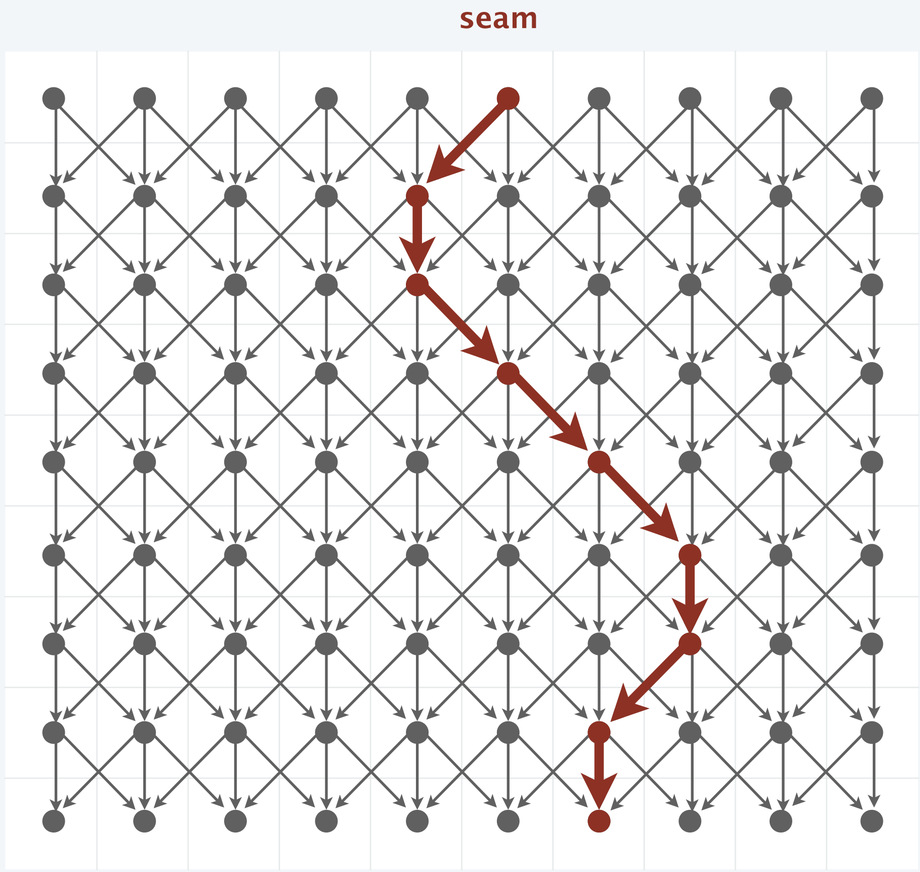
Content-aware resizing: dynamic programming formulation

Problem. Find a min energy path from top to bottom.

Subproblems. $distTo(col,\ row)$ = energy of min energy path from any top pixel to pixel (col, row).

Goal. $\min \{\ distTo(col, H-1)\ \}$.

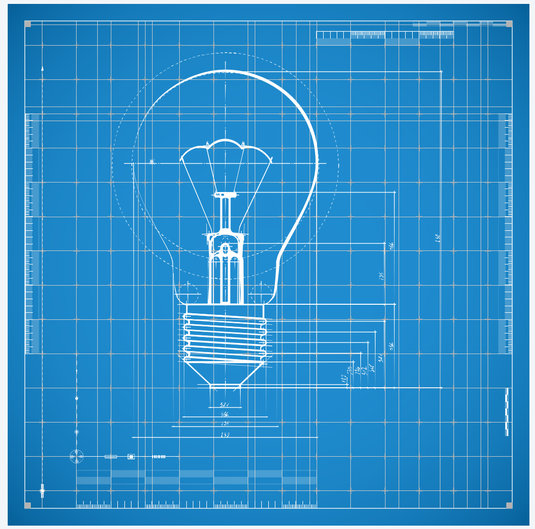
Dynamic programing recurrence. For you to figure out in Assignment 6.



Summary

How to design a dynamic programming algorithm.

- Find good subproblems. 
- Develop DP recurrence for optimal value.
  - optimal substructure
  - overlapping subproblems
- Determine dependency order in which to solve subproblems.
- Cache computed results to avoid unnecessary re-computation.
- Reconstruct the optimal solution via backtracing.



Credits

image	source	license
Richard Bellman	Wikipedia	
Biopython	biopython.org	
ImageMagick Liquid Rescale	ImageMagick	ImageMagick license
Cubic B-Spline	Tibor Stanko	
Leonardo Fibonacci	Wikimedia	public domain
Evoke 5 Vending Machine	U-Select-It	
U.S. Coins	Adobe Stock	education license
Seam Carving	Avidan and Shamir	
Broadway Tower	Wikimedia	CC BY 2.5
A is for Algorithms	Reddit	

A final thought